FRE-GY 9743 — Assignment 5

Implicit Function Differentiation and a Fixed-Cashflow Valuation Engine

Yuri Moghaddam

October 2026

Repository (computational part: discount-factor gradients and the fixed-accrued valuation engine):
https://github.com/Yuri48/FRE-GY-9743-Assignments-5/tree/Yuri_Moghaddam_Branch
The notebook `hw5_valuation_and_first_order_risk.ipynb` runs end to end on the completed `fixedincomelib` (TODOs 1–5: the local and the recursive discount-factor gradient, and the constructor, value and risk of `ValuationEngineProductFixedAccrued`). Its last section checks every analytic gradient against bump-and-reprice, the product-rule decomposition, the `scaler` and `accumulate` semantics, and the same-day, past and short-notional cases. Part II below summarises the implementation and the numbers.

Part I. Implicit function differentiation

The setting. A model $M(X^I)$ is driven by an internal state $X^I \in \mathbb{R}^n$: IFR node values, SABR parameters, the θ of a smile interpolator. The market supplies n observables $X^M \in \mathbb{R}^n$ (par swap rates, option prices), and for each one the model has a pricing function $V_j(X^I)$. Calibration is finding the state that reproduces the market,

$$V(X^I) = X^M \quad \Longleftrightarrow \quad R(X^I) := V(X^I) - X^M = 0,$$

a square system of n nonlinear equations in n unknowns. The state is what we solve for; the market is what we are given. All three questions are about the map between the two and its inverse.

Q1. Recipe for solving $V(X^I) = X^M$.

1. **Make it square and well posed.** Choose the parametrization so that $\dim X^I = \dim X^M$, every V_j is smooth in X^I, and the Jacobian $\mathbf{J} = \partial V/\partial X^I$ is nonsingular on the region of interest. That is what makes the solution exist and be unique, and without it there is nothing to differentiate in Q2. Decide which space to solve in: raw prices, or a transformed quote (implied vol, par rate, zero rate) in which V is closer to linear and the residual is in units a trader recognises.
2. **Start close.** Yesterday's calibrated state, or a closed-form approximation (zero rate $\approx$ par rate; ATM vol $\approx \alpha F^{\beta-1}$ for SABR). Newton converges quadratically, but only locally; the starting point is where most calibration failures come from.
3. **Newton–Raphson.** Linearise the residual and solve one linear system per step,

$$X^I_{k+1} = X^I_k - \mathbf{J}(X^I_k)^{-1} R(X^I_k), \qquad \mathbf{J}_{ij} = \frac{\partial V_i}{\partial X^I_j}.$$

Each iteration prices the n calibration instruments once and evaluates $\mathbf{J}$ once: analytically through the chain rule (the discount-factor gradients of Part II are exactly this, $\partial D/\partial f_i = -D\,w_i$), by algorithmic differentiation, or by finite differences when nothing better is available. Stop when $\|R\|$ is below a tolerance in the chosen space (10^{-10} in price, 10^{-8} in rate) or the step is negligible.

4. **Safeguard.** Damp the step (a line search on $\|R\|$) so it never increases the residual; keep the parameters in their admissible domain (positive vols, $|\rho| < 1$); if $\mathbf{J}$ becomes nearly singular switch to Levenberg–Marquardt, $(\mathbf{J}^\top\mathbf{J} + \lambda I)\,\delta = -\mathbf{J}^\top R$, which interpolates between Newton and steepest descent.
5. **Exploit structure.** When instrument j depends only on the first j state variables (curve instruments ordered by maturity with a local interpolator, such as the piecewise-constant one here), $\mathbf{J}$ is lower triangular and the n-dimensional Newton step collapses into n one-dimensional root searches solved in sequence. That is the bootstrap: solve the first node from the first instrument, freeze it, solve the second, and so on. A non-local interpolator (a cubic spline) makes every instrument depend on every node, $\mathbf{J}$ is dense, and one needs the full multidimensional Newton with a few global sweeps.
6. **Check.** Re-price the calibration set to tolerance and look at the condition number of $\mathbf{J}$ at the solution. A huge condition number means the market inputs barely identify the state: the calibration will "succeed", but the state, and every sensitivity derived from it, will be noise.

With more instruments than parameters the root search becomes a weighted least-squares problem,

$$\min_{X^I} \|W^{1/2} R(X^I)\|^2,$$

solved by Gauss–Newton or Levenberg–Marquardt; with fewer, a regulariser (smoothness, distance to a prior) makes the system square again. The square exact-fit case is what the assignment asks about and what the rest of this part assumes.

Q2. $\mathrm{d}X^I/\mathrm{d}X^M$ **at the solution.** At the optimum $X^I_\star$ the fit is exact, $V(X^I_\star) = X^M$, and the question is how $X^I_\star$ moves when the market moves. The implicit function theorem answers it without re-solving anything: if $\mathbf{J} = \partial V/\partial X^I$ is invertible at $X^I_\star$, then near X^M the solution is a smooth function $X^I(X^M)$, and differentiating the identity $V(X^I(X^M)) \equiv X^M$ with respect to X^M gives

$$\mathbf{J}\,\frac{\mathrm{d}X^I}{\mathrm{d}X^M} = I \qquad \Longrightarrow \qquad \frac{\mathrm{d}X^I}{\mathrm{d}X^M} = \mathbf{J}^{-1} = \Big(\frac{\partial V}{\partial X^I}\Big)^{-1}\Big|_{X^I_\star}.$$

Three remarks.

- **It is free.** $\mathbf{J}$ was already evaluated, and factorised, in the last Newton iteration; one reuses the LU factors and solves, and never forms the inverse explicitly. The alternative, bumping each market input and recalibrating, costs n extra calibrations, each polluted by the convergence tolerance divided by the bump size. The implicit-function derivative is the exact derivative of the exact solution, consistent with the calibration to machine precision.
- **It turns state risk into market risk.** For any product with value $P(X^I)$ the chain rule gives

$$\frac{\partial P}{\partial X^M} = \frac{\partial P}{\partial X^I}\,\frac{\mathrm{d}X^I}{\mathrm{d}X^M} = \nabla_{X^I} P\;\mathbf{J}^{-1}.$$

  The row vector $h = \nabla_{X^I} P\,\mathbf{J}^{-1}$ is also the hedge: row j of $\mathbf{J}$ is the state risk $\nabla_{X^I} V_j$ of calibration instrument j, so the portfolio $\sum_j h_j V_j$ has state risk $h\mathbf{J} = \nabla_{X^I} P$, the same as the product. Par-rate DV01s, vega by market option and hedge ratios are one object. In the library this is the step

from the engine's raw blocks $\nabla_{X^I}\mathrm{PV}$ to the risk report, and the model carries a `model_jacobian` slot for $\mathbf{J}$. For the direct IFR inputs of this assignment V is the identity, $\mathbf{J} = I$, and the raw node derivatives already are the market risk, which is why the notebook says the state-to-quote conversion is the identity; with a bootstrapped curve the same report would multiply the raw blocks by $\mathbf{J}^{-1}$.

- **Structure carries over.** If $\mathbf{J}$ is lower triangular (bootstrap), so is $\mathbf{J}^{-1}$: a bump to the 5Y par rate moves the 5Y node and every node beyond it, none before. If $\mathbf{J}$ is ill conditioned, $\mathbf{J}^{-1}$ amplifies it: the sensitivities become large and offsetting, correct to first order and useless for hedging, and the cure is a better parametrization (Q3), not more digits. For a least-squares fit the same argument applied to the normal equations gives $\mathrm{d}X^I/\mathrm{d}X^M = (\mathbf{J}^\top W \mathbf{J})^{-1}\mathbf{J}^\top W$, and differentiating the identity once more gives the curvature $\mathrm{d}^2X^I/\mathrm{d}(X^M)^2$ if it is ever needed.

Q3. Three OTM options and a vol interpolator. *The map.* The market gives undiscounted prices P_1, P_2, P_3 of an OTM put at K_1, the ATM option at K_{atm} and an OTM call at K_3, all for one expiry T and forward F. Undiscounted means the discount curve has been divided out already, so the pricing function is the undiscounted Black formula on the forward,

$$B(K,\sigma) = \omega\big[F\,N(\omega d_1) - K\,N(\omega d_2)\big], \qquad d_{1,2} = \frac{\ln(F/K) \pm \tfrac{1}{2}\sigma^2 T}{\sigma\sqrt{T}},$$

with $\omega = +1$ for the call and -1 for the put, and the only thing left to calibrate is the volatility, which the interpolator $\sigma(K;\theta)$ supplies. In the language of Q1 and Q2,

$$X^M = (P_1, P_2, P_3), \qquad X^I = \theta \in \mathbb{R}^3, \qquad V_i(\theta) = B\big(K_i, \sigma(K_i;\theta)\big).$$

Three prices identify three parameters, so θ must have exactly three components for the system to be square; the quadratic-in-log-moneyness and the SABR (α, ρ, ν) choices discussed below both do.

Solving for θ (Q1). The Jacobian factorises by the chain rule,

$$\mathbf{J}_{ij} = \frac{\partial B_i}{\partial \sigma}\,\frac{\partial \sigma(K_i;\theta)}{\partial \theta_j} = \nu_i\, S_{ij}, \qquad \nu_i = F\sqrt{T}\,\varphi\big(d_1^{(i)}\big) > 0, \qquad S = \frac{\partial \sigma}{\partial \theta}\Big|_{K_1,\,K_{\mathrm{atm}},\,K_3},$$

ν_i being the undiscounted vega and S the 3×3 sensitivity of the interpolator at the three quoted strikes. So $\mathbf{J} = \mathrm{diag}(\nu)\,S$, and $\mathbf{J}$ is invertible exactly when S is, i.e. when the three strikes identify the three parameters. In practice one solves in two stages, which is the same system with the diagonal vega factor removed:

1. invert each price separately for its implied vol, $B(K_i, \sigma_i^{\mathrm{mkt}}) = P_i$, a one-dimensional Newton or Brent search. The root is unique because B is strictly increasing in σ, and well conditioned because an OTM option is all time value, so its vega is far from zero. That is why the market quotes OTM options and why the problem is posed with them;
2. solve $\sigma(K_i;\theta) = \sigma_i^{\mathrm{mkt}}$, $i = 1, 2, 3$, by Newton with Jacobian S, or in closed form when the interpolator is linear in θ. The residual is now in vol units, which is how a trader judges a fit.

The two stages give the same θ as Newton on the prices directly, because $\mathrm{diag}(\nu)$ is invertible. Uniqueness is the invertibility of S: for $\sigma(K;\theta) = \theta_0 + \theta_1 x + \theta_2 x^2$ with $x = \ln(K/F)$, S is a Vandermonde matrix in $(x_1, x_{\mathrm{atm}}, x_3)$ and is invertible for any three distinct strikes; for SABR with β fixed it is invertible as long as the wings are far enough from the money that the curvature is actually observed.

Sensitivities (Q2). Two different derivatives are being asked for, and only one needs the implicit function theorem.

- *How $\sigma(K)$ changes with θ.* That is $S(K) = \partial\sigma(K;\theta)/\partial\theta$, a direct derivative of the interpolator at any strike K, quoted or not: $(1, x, x^2)$ for the quadratic, the derivative of Hagan's expansion in (α, ρ, ν) for SABR. No inversion is involved; this is the forward map.
- *How θ, and with it the whole smile, changes with the market.* By Q2, $\mathrm{d}\theta/\mathrm{d}P = \mathbf{J}^{-1} = S^{-1}\,\mathrm{diag}(1/\nu)$, and for any strike

$$\frac{\mathrm{d}\sigma(K)}{\mathrm{d}P} = S(K)\,S^{-1}\,\mathrm{diag}(1/\nu), \qquad \frac{\mathrm{d}\sigma(K)}{\mathrm{d}\sigma^{\mathrm{mkt}}} = S(K)\,S^{-1}.$$

  The second expression is the set of interpolation weights of the smile: for the quadratic they are the Lagrange basis polynomials through the three strikes, they sum to one, and a parallel move of the three market vols moves every $\sigma(K)$ in parallel. For an exotic with value $E(\theta)$ the same step gives $\mathrm{d}E/\mathrm{d}P = (\partial E/\partial\theta)\,\mathbf{J}^{-1}$: the number of each of the three market options that hedges the exotic's smile risk to first order, from one linear solve and no recalibration.

What θ should mean to a trader. Three OTM prices are one point on the put wing, one at the money and one on the call wing. The three things such a triple can say are the level, the slope and the curvature of the smile, and the parametrization should say them in those words:

Parameter	Effect on $\sigma(K)$	Market quote	Hedge / risk bucket
θ_0: level	parallel shift, $\partial\sigma/\partial\theta_0 = 1$	ATM vol σ_{atm}	ATM straddle: vega
θ_1: skew	tilt around ATM, $\propto x$: put wing up, call wing down	risk reversal $\mathrm{RR} = \sigma_{K_3} - \sigma_{K_1}$	risk reversal: long call, short put
θ_2: convexity	both wings up against ATM, $\propto x^2$	butterfly $\mathrm{BF} = \frac{1}{2}(\sigma_{K_1} + \sigma_{K_3}) - \sigma_{\mathrm{atm}}$	butterfly: wings against the straddle

This is the FX market's $(\sigma_{\mathrm{atm}}, \mathrm{RR}, \mathrm{BF})$ convention, and it is also what the SABR triple means in practice: α sets the level, ρ the skew, ν the convexity. With θ chosen this way the columns of S are a constant, an odd and an even function of log-moneyness, nearly orthogonal, so S is well conditioned and $\mathrm{d}\sigma(K)/\mathrm{d}\theta$ reads as "vega, skew risk, convexity risk": the three buckets a vol desk already hedges with straddles, risk reversals and butterflies. The same smile parametrised by polynomial coefficients in K, or by three spline knots, fits the same three prices with the same curve, but the columns of S are then nearly collinear, $\mathbf{J}^{-1}$ is large and oscillating, and the sensitivities are right and unreadable at the same time. The rule is to make θ the things one can trade. Two smaller points follow from the way the question is posed: using OTM options keeps every ν_i large, so $\mathbf{J}$ is as well conditioned as the strikes allow; using undiscounted prices keeps the discount curve out of θ altogether, so the vol parameters carry only vol information and their sensitivities have no rates leg.

Part II. The valuation engine

What the five TODOs compute. Each curve component c stores IFR node values $f^{(c)}$ on a piecewise-constant interpolator, and its local discount factor is $D_c(T) = \exp\big(-\int_0^T f^{(c)}(s)\,\mathrm{d}s\big)$. The integral is linear in the node values, with weights $w_i = \partial\big(\int_0^T f\big)/\partial f_i$ equal to the length of segment i inside $[0, T]$ (the interpolator supplies them), so

$$\text{TODO 1:} \qquad \frac{\partial D_c(T)}{\partial f_i^{(c)}} = -D_c(T)\,w_i.$$

The model discount factor of a funding curve follows its reference chain, $D_{\text{funding}} = D_{\text{spread}}\, D_{\text{SOFR}}$, and the product rule gives one block per component,

$$\text{TODO 2:} \qquad \nabla_{f^{(\text{SOFR})}} D_{\text{funding}} = D_{\text{spread}}\, \nabla D_{\text{SOFR}}, \qquad \nabla_{f^{(\text{spread})}} D_{\text{funding}} = D_{\text{SOFR}}\, \nabla D_{\text{spread}},$$

both equal to $-D_{\text{funding}}\, w$ with the respective weights. The implementation walks the chain recursively, passing down the product of the factors above a component and receiving back the product of the factors below it, so it works for chains of any depth, and components outside the chain get zero. The engine (TODOs 3–5) reads the funding index from the `FUNDING INDEX PARAMETER`, forms the signed cashflow $C = N\alpha$ from the product's notional and accrual fraction, and sets

$$\text{PV} = C\, D_{\text{funding}}(T_p), \qquad \nabla\text{PV} = C\, \nabla D_{\text{funding}}(T_p)$$

for a future payment, PV = cash = C with zero risk for a payment today, and zero for a payment in the past. In every gradient method `scaler` multiplies the new contribution and `accumulate` decides whether it replaces or adds to the container; the engine simply forwards `scaler` $\times\, C$ to the model.

Numbers. Value date 2026-01-02, payment date 2026-10-02 ($T_p = 0.747945$ years ACT/ACT), notional 1,000,000 accruing ACT/360 from 2026-04-02 to 2026-07-02, so that $\alpha = 0.252778$ and $C = 252{,}777.78$. The SOFR IFR nodes are 3%, 4%, 5% at 2026-04-02, 2027-01-02, 2028-01-02; the funding spread nodes are 0 and 10 bp at 2026-07-02 and 2027-01-02.

	SOFR-1B	SOFR-1B-FLAT, local (spread)	SOFR-1B-FLAT, model
Discount factor to T_p	0.972921	0.999748	0.972676

PV = 245,870.91, cash today = 0. The raw derivatives $\partial\text{PV}/\partial f_i$ per unit of IFR, and the report's view per +1 bp node shift:

Component	Node date	w_i (years)	$\partial\text{PV}/\partial f_i$	per +1 bp
SOFR-1B	2026-04-02	0.246575	−60,625.70	−6.0626
SOFR-1B	2027-01-02	0.501370	−123,272.26	−12.3272
SOFR-1B	2028-01-02	0.000000	0.00	0.0000
SOFR-1B-FLAT	2026-07-02	0.495890	−121,925.03	−12.1925
SOFR-1B-FLAT	2027-01-02	0.252055	−61,972.94	−6.1973

The third SOFR node starts a segment after the payment date, so its weight and derivative are zero. Every other derivative is negative: a higher forward rate lowers the discount factor of a positive cashflow. The two blocks are −PV times the segment lengths, which is the product rule in its simplest form.

Checks. The notebook's last section bumps each IFR node by $\pm 10^{-5}$ (through the model's `perturb_model_parameter`), reprices the component, the model and the engine, and compares central differences with the analytic gradients (largest discrepancy 1.2×10^{-6} in $\partial\text{PV}/\partial f$, against derivatives of order 10^5); verifies $D_{\text{funding}} = D_{\text{SOFR}} D_{\text{spread}}$ and the two product-rule blocks; verifies that a call with `scaler` equal to 2, followed by one with `scaler` equal to 3 and `accumulate` on, gives five times the base gradient, and that an overwrite resets it, at the component, model and product level; and verifies the same-day case (PV = cash = C, zero risk), the past case (all zero, with an accumulated gradient left untouched) and a short position (PV and risk flip sign).